Listing 4.6 Example creating an AI provider

```
const AI = createAI({
  actions: {                                          Defines your server actions
    fetchUserData: async (userId) => {
      const response = await fetch(`/api/users/${userId}`);
      const data = await response.json();
      return data;
    },
    updateUserSettings: async (settings) => {
      await fetch('/api/settings', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(settings),
      });
    }
  },
  initialAIState: {
    userProfile: { name: '', email: '' },
    settings: { theme: 'light', notifications: true },
  },                                                  Defines an initial AI state

  initialUIState: {
    isSidebarOpen: false,
  },                                  Defines an initial UI state
});
...
const AppLayout = ({ children }) => {
  return (
    <AI>.                             Wraps the AI component to the root layout
      {children}                      of the app so that the context is available
    </AI>                             throughout the children components
  )
 }
```

The `createAI` function returns a React context provider component with the defined actions: initial AI and UI states. You will need to place this component on your app layout page so that the hooks discussed after the following sidebar can access the state and actions.

Saving and restoring AI state with createAI

The `createAI` function offers a few useful callbacks for managing AI state persistence. Use the `onSetAIState` callback to save AI state changes to a database—for example:

```
onSetAIState: async ({ state, done }) => {
  if (done) {
    await saveToDatabase(state);
  }
}
```